sBÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Trần Lê Minh Tâm
MSSV / Lớp	3124720064 / DNT1241

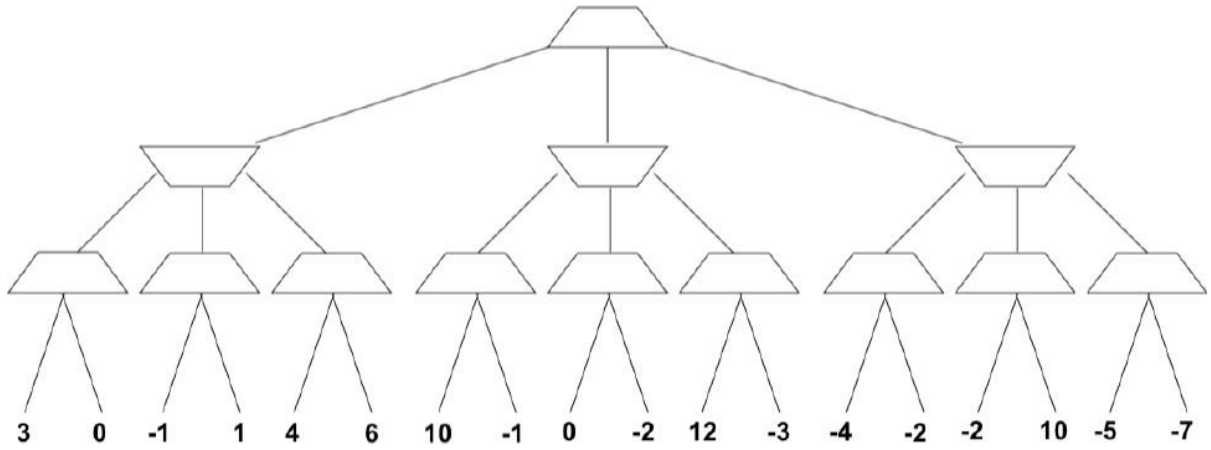
Mục tiêu học tập

- Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa.
- Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
- Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP.
- Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi.
- Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

Tầng sát lá (MAX) từ trái sang phải: max(3, 0) = 3; max(-1, 1) = 1; max(4, 6) = 6 | max(10, -1) = 10; max(0, -2) = 0; max(12, -3) = 12 | max(-4, -2) = -2; max(-2, 10) = 10; max(-5, -7) = -5.

Tầng giữa (MIN) từ trái sang phải: min(3, 1, 6) = 1; min(10, 0, 12) = 0; min(-2, 10, -5) = -5.

Nút gốc (MAX): max(1, 0, -5) = 1.

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

MAX nên chọn nhánh ngoài cùng bên trái (nhánh 1). Giá trị trò chơi là 1.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Tại nút MIN thứ nhất (trái): $\alpha = -\infty$, $\beta = 1$. Thăm hết các lá bên trái.

Tại nút MIN thứ hai (giữa): $\alpha = 1$, $\beta = 0$ (sau khi thăm nhánh 0, -2 và có giá trị 0). Vì $\beta \leq \alpha$ (0 < 1), cắt tỉa nhánh MAX cuối cùng của cụm giữa (có các lá 12, -3).

Tại nút MIN thứ ba (phải): $\alpha = 1, \beta = -2$ (sau khi thăm nhánh -4, -2 và có giá trị -2). Vì $\beta \leq \alpha$ (-2 < 1), cắt tỉa toàn bộ hai nhánh MAX còn lại của cụm phải (chứa các lá -2, 10 và -5, -7).

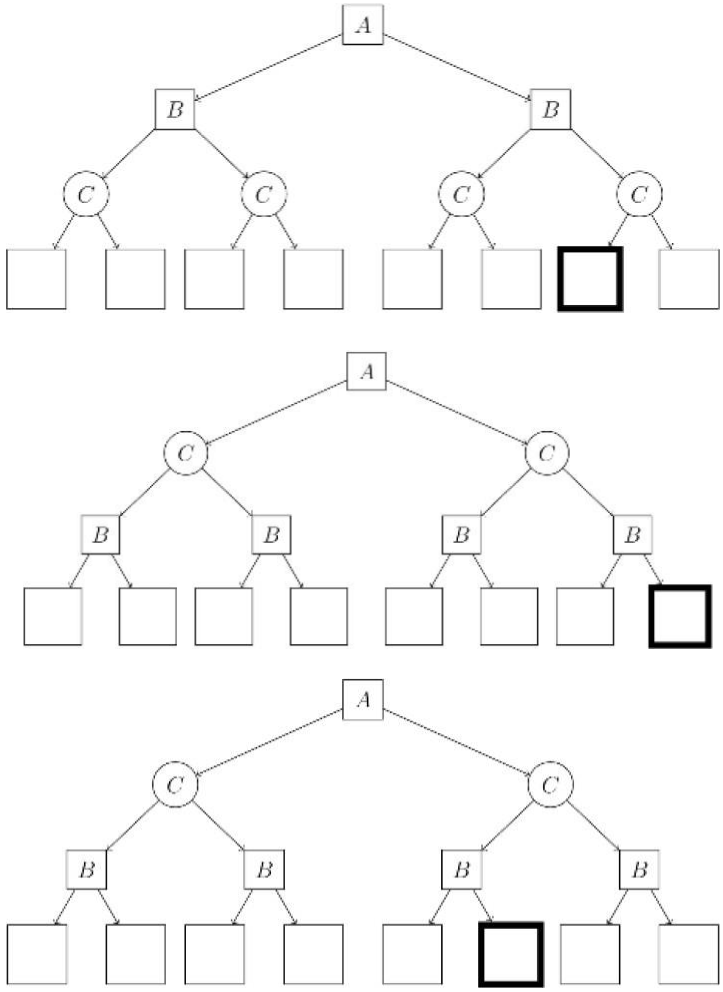
d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

Các nút lá không cần thăm: `(12, -3), (-2, 10), (-5, -7).`

Minimax đánh giá toàn bộ 18 nút lá. Alpha-Beta chỉ đánh giá 12 nút lá (cắt tỉa 6 nút).

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Điền lá: Cho các lá của cụm trái đều bằng 0, L_B = 0. Ở nhánh phải, cho hai lá đầu của cụm C bên trái bằng 10, 10.

Điều kiện cắt tỉa: B là nút MAX nên R_B sẽ có giá trị ≥ 10. Vì A là nút MIN và đã có nhánh trái (L_B) = 0, nhánh phải R_B ≥ 10 chắc chắn lớn hơn 0, nên A sẽ không bao giờ chọn nhánh phải. Nút viền đậm có thể bị cắt tỉa mà không ảnh hưởng đến quyết định của A.

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Điền lá: Cụm L_C có các lá bằng 0 → L_C = 0. Cụm R_C có L_B = 20 (lá bằng 20). Nút lá đầu tiên của R_B là 0.

Điều kiện cắt tỉa: Do R_B là nút MAX, R_B ≥ 0. C là nút ngẫu nhiên nên giá trị kỳ vọng R_C = $0.5 \times 20 + 0.5 \times R_B \geq 10$. A là nút MIN, so sánh L_C (0) và R_C (≥ 10) sẽ ưu tiên L_C. Do đó, nút lá viền đậm (thuộc R_B) không cần duyệt.

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Không thể
Để cắt tỉa lá thứ 2 của L_B (nút đậm), ta cần chứng minh R_C luôn lớn hơn hoặc nhỏ hơn L_C. Tuy nhiên R_C = $0.5 \times L_B + 0.5 \times R_B$. Nếu không có giới hạn dưới (lower bound) cho các nút lá, nhánh R_B (chưa thăm) hoàn toàn có thể mang giá trị âm vô cùng, kéo kỳ vọng của R_C xuống thấp hơn L_C. Do đó, buộc phải thăm hết để tính giá trị chính xác của C trừ khi biết trước biên độ của lá.

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Trong Minimax, các nút MIN/MAX tự động tạo ra cận dưới/cận trên cho chính nó. Trong cây ngẫu nhiên, nút ngẫu nhiên tính giá trị trung bình, do đó một nút con chưa thăm với giá trị cực đoan có thể thay đổi hoàn toàn giá trị kỳ vọng của nút cha. Do đó, không thể cắt tỉa tại nút ngẫu nhiên nếu không biết khoảng giá trị (bounds) của các lá.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \leq V_E$. X Đúng ☐ Sai

Minimax giả định đối thủ chọn nước đi tồi tệ nhất (lấy Min), trong khi Expectimax tính trung bình các nước đi của đối thủ. Trung bình của một tập hợp luôn lớn hơn hoặc bằng giá trị nhỏ nhất của tập đó.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. X Đúng ☐ Sai

Chính sách Minimax tối ưu bảo đảm mức lợi ích thấp nhất (worst-case) là V_M đối với một đối thủ hoàn hảo. Nếu đối thủ chơi ngẫu nhiên, kết quả ở mỗi lá vẫn luôn $\geq V_M$, do đó giá trị kỳ vọng chắc chắn $\geq V_M$.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng X Sai

V_E là kỳ vọng cực đại thu được khi ta chơi tối ưu hóa riêng cho Expectimax. Minimax có thể bỏ qua một nhánh có kỳ vọng rất cao (giá trị V_E) chỉ vì nhánh đó chứa một lá rủi ro duy nhất, dẫn đến lợi ích thấp hơn V_E.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

☐ Minimax với tiện ích trạng thái cuối +1/−1

X Expectimax với tiện ích trạng thái cuối +1/−1

☐ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

☐ Không phương án nào

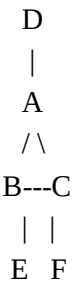
d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

Vì kết quả ngẫu nhiên và không có đối thủ (trò chơi 1 người), ta cần mô hình hóa sự bất định bằng xác suất. Expectimax hợp lý nếu cây trạng thái hữu hạn và không có chu trình. Value Iteration là mô hình giải Quyết định Markov (MDP) tiêu chuẩn nếu coi đây là môi trường đồ thị có trạng thái lặp và thỏa mãn tính Markov. Minimax bị loại vì không có tác tử đối kháng.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Miền của các biến kề trực tiếp với A là B, C, D.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

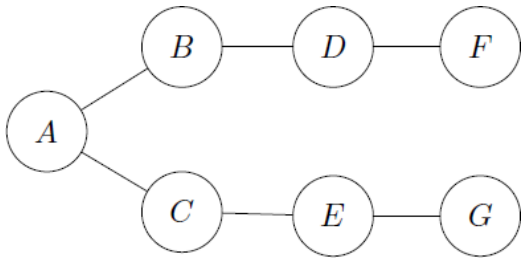
FC chỉ kiểm tra các biến kề chưa được gán. Kề với B có A (đã gán), C và E. Do đó, miền bị thay đổi là C, E.

d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?
AC lan truyền toàn mạng. Gán A thu hẹp B, C, D. B thay đổi kéo theo E, C thay đổi kéo theo B và F. D không nối với biến chưa gán nào khác. Vậy miền bị thay đổi gồm: B, C, D, E, F (tất cả các biến còn lại).

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

Khi gán B, nó trực tiếp giới hạn C và E. Sự thay đổi của C lan truyền qua cung (F, C) ảnh hưởng đến F. D hoàn toàn độc lập với B, C, E, F sau khi A đã gán, nên không đổi. Vậy miền thay đổi gồm: C, E, F

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

f) Thứ tự A → B → C → D → E → F → G\

số lần backtrack xấu nhất = **0**.

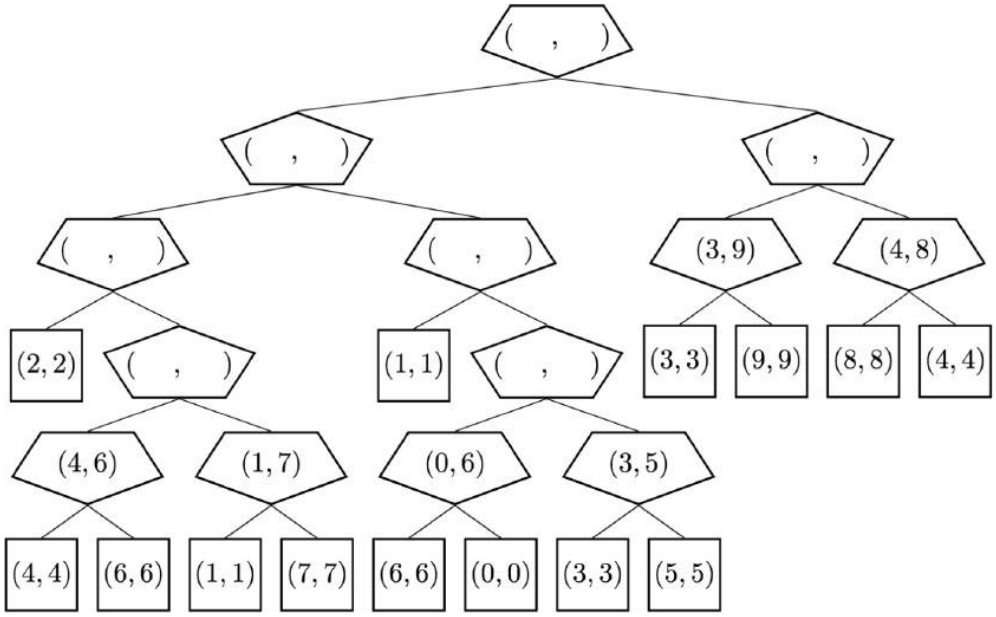
Đây là đồ thị dạng cây và thứ tự gán này chính là thứ tự sắp xếp tô-pô (từ gốc A đến lá). Thuật toán chạy AC ngay từ đầu (làm cho cây đạt Arc Consistency hướng tâm). Với cây đã có AC, mọi phép gán xuôi theo chiều tô-pô đều đảm bảo luôn tìm được giá trị hợp lệ mà không bao giờ cần quay lui (backtrack-free).

g) Thứ tự G → E → C → A → B → D → F\
Số lần backtrack xấu nhất = **0**.
Nếu coi G là gốc, đồ thị này vẫn là cây. Thứ tự này hoàn toàn tương đương với một thứ tự duyệt tô-pô hợp lệ xuất phát từ gốc G. Do đó tiếp tục không có backtrack.

h) Thứ tự G → F → E → D → C → B → A
Số lần backtrack lớn.
G và F là hai lá ở hai nhánh độc lập. Gán G và F ở hai bước đầu không kích hoạt AC kịp thời để kiểm tra tính tương thích giữa chúng. AC chỉ chạy ở các bước chẵn (sau G, F và sau E, D...). Việc gán các biến độc lập phá vỡ cấu trúc tô-pô, có thể dẫn đến việc G và F giới hạn miền của A về các tập hợp giao nhau rỗng, gây ra lỗi xung đột cục bộ. Xấu nhất phải backtrack qua lại nhiều tổ hợp của các nhánh rời rạc (cỡ $O(d^2)$ hoặc hơn tùy thời điểm phát hiện).

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper. Các nút MAX (Pacman, ngũ giác hướng Lên) chọn nhánh có u lớn nhất. (u_node, v_node) bằng đúng (u, v) của nhánh được chọn.

Các nút MIN (Casper, ngũ giác hướng Xuống): $u = \min(u_{con})$ (vì Pacman tin Casper giảm tiện ích), nhưng $v = \max(v_{con})$ (vì Casper thực chất muốn giúp và chọn nhánh tối đa hóa v).

Nút Pacman E (ngũ giác trống sát lá 4,6): chọn giữa F(4,6) và G(1,7) → Max u là 4 → điền (4, 6).

Nút Pacman H (ngũ giác trống sát lá 0,6): chọn giữa I(0,6) và J(3,5) → Max u là 3 → điền (3, 5).

Nút Casper C (bên trái tầng 2): con là (2,2) và E(4,6) → $u = \min(2,4) = 2$, $v = \max(2,6) = 6$ → điền (2, 6).

Nút Casper D (bên phải nhánh trái): con là (1,1) và H(3,5) → $u = \min(1,3) = 1$, $v = \max(1,5) = 5$ → điền (1, 5).

Nút Pacman A (tầng 1 trái): con là C(2,6) và D(1,5) → Max u là 2 → điền (2, 6).

Nút Pacman B (tầng 1 phải): con là K(3,9) và L(4,8) → Max u là 4 → điền (4, 8).

Nút Casper Root (Gốc): con là A(2,6) và B(4,8) → $u = \min(2,4) = 2$, $v = \max(6,8) = 8$ → điền (2, 8).

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Theo thứ tự trái sang phải, khi Casper tại nút A biết $u_C = 2$, Casper bắt đầu duyệt D. Vừa gặp lá (1,1) đầu tiên, Casper suy ra $u_D \leq 1$. Do $1 < 2$, Pacman tại A sẽ không bao giờ chọn D. Vậy Casper không cần duyệt nút H (và toàn bộ con của H). Tương tự, tại E (Pacman), sau khi thấy nhánh trái là 4, gặp lá (1,1) của nhánh phải G, Pacman thấy G có $u \leq 1 < 4$, nên Casper có thể bỏ qua nút lá (7,7).

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Pacman và Casper cùng chia sẻ một mục tiêu là tối đa hóa tiện ích. Toàn bộ cây trở thành một cây MAX hoàn toàn (không còn sự khác biệt giữa u và v). Giá trị của trò chơi sẽ bằng tiện ích của nút lá lớn nhất trên cây, tức là **9**.

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Branching factor (b) hữu hạn và nghiệm nằm ở độ sâu (d) hữu hạn.	Mọi bước đi đều có chi phí bằng nhau (hoặc là hàm không giảm theo độ sâu).

Depth-First Search (DFS)	Không gian trạng thái hữu hạn và không có chu trình (vòng lặp).	Không bao giờ được đảm bảo (chỉ trả về nghiệm đầu tiên chạm đến).
Iterative Deepening Search (IDS)	Branching factor (b) hữu hạn.	Tương tự BFS, chi phí các cạnh (bước đi) phải đồng nhất.
A* Search	Branching factor hữu hạn và chi phí mỗi cạnh luôn $\geq \epsilon > 0$.	Heuristic hàm $h(n)$ phải là Admissible (Chấp nhận được, không đánh giá quá mức) đối với tìm kiếm trên cây; và phải Consistent (Nhất quán) đối với tìm kiếm trên đồ thị

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

Nếu chi phí cạnh bằng 0 hoặc âm, thuật toán có thể mắc kẹt trong vòng lặp vô hạn mà chi phí không tăng (làm mất tính Đầy đủ). Về mặt heuristic, nếu đánh giá sai lệch quá cao so với thực tế, A* có thể "sợ hãi" bỏ qua nhánh chứa nghiệm tối ưu thực sự và chọn đường đi dài hơn (làm mất tính Tối ưu).

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Chi phí đường tối ưu cũ tăng lên, có thể làm nó không còn tối ưu nữa. Cần đưa các nút bị ảnh hưởng trở lại hàng đợi OPEN và lan truyền chi phí mới (như thuật toán D*) để xem có đường đi thay thế rẻ hơn không.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Đường này vốn dĩ đã tối ưu, nay lại càng rẻ hơn. Nó chắc chắn vẫn là đường tối ưu tuyệt đối. Không cần duyệt lại hay cập nhật thuật toán, chỉ cần ghi nhận tổng chi phí mới giảm đi n.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Đường tối ưu hiện tại không chứa cạnh này nên không bị ảnh hưởng. Cạnh đắt lên càng khiến nó không có cơ hội được chọn. Không cần làm gì cả.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Sự giảm giá có thể biến đường này thành tối ưu mới nếu mức giảm đủ sâu. Cần tính lại hàm f qua cạnh này. Nếu $f_{mới} < f^*$ (chi phí tối ưu cũ), đẩy nút Y vào lại OPEN và tiếp tục chạy A*

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Cạnh này chưa từng được xét vì hàm f ban đầu của nó đã quá kém. Việc tăng chi phí chỉ làm nó tồi tệ hơn. Không cần làm gì cả.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Tương tự trường hợp giảm n ở nhánh đã khám phá. Cần đánh giá lại $f = g(X) + (c - n) + h(Y)$. Nếu f rớt xuống ngưỡng tốt hơn kết quả hiện tại, ta mở rộng nút Y và chạy tiếp A* từ đó.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

• A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Lớp tác tử **A**. Vì quan sát đầy đủ và xác định, toàn bộ thông tin cần để ra quyết định đều nằm ở tín hiệu cảm biến (percept) hiện tại. Tác tử chỉ cần một hàm phản xạ không trạng thái $f_A(\ \ )$ là có thể hoàn hảo.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Lớp tác tử **C**. Tầm nhìn bị che khuất một phần buộc tác tử phải dùng lịch sử các cảm biến $f_C(percepts)$ để ghi nhớ quá khứ, nội suy trạng thái hiện tại thực sự là gì.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Lớp tác tử **C**. Vừa mù mờ về thông tin, vừa có yếu tố ngẫu nhiên làm thay đổi thế giới. Tác tử bắt buộc phải phân tích chuỗi percepts lịch sử để duy trì tập niềm tin (Belief State) về các khả năng có thể xảy ra và chọn hành động tốt nhất.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện

Đúng. Ví dụ thực tế: Một môi trường "tĩnh lặng tuyệt đối" không có hình phạt và phần thưởng luôn bằng 0 cho mọi hành động (hoặc phần thưởng bằng hằng số tối đa ở mọi trạng thái). Khi mọi kết quả trả về luôn giống hệt nhau, mọi quyết định dù ngẫu nhiên hay cứng nhắc (của bất kì lớp tác tử nào) đều được định nghĩa toán học là hành vi tối ưu/hợp lý.

— Hết —